

Proyecto Unidad III

Auditoría automatizada de bases de datos para reportes empresariales

Denis Javier Rivera Arbaiza - RA21117

Proyecto Unidad III

Auditoría automatizada de bases de datos para reportes empresariales

1. Contexto del problema

En una empresa de análisis de datos es común recibir varias bases de datos provenientes de distintas áreas: ventas, transporte, producción, inventarios o estudios técnicos.

Antes de hacer análisis avanzados, es necesario revisar la calidad de cada base de datos. Por ejemplo:

- identificar qué columnas son numéricas;
- detectar valores faltantes;
- calcular medidas resumen;
- revisar si hay columnas con poca variabilidad;
- encontrar posibles errores;
- comparar variables entre sí;
- generar reportes automáticos.

En este proyecto se construirá un sistema sencillo de auditoría de datos usando únicamente herramientas de iteración en R.

2. Bases de datos a utilizar

Usaremos bases de datos conocidas que vienen incluidas en R:

```
data(mtcars)
data(cars)
data(iris)
data(airquality)
```

Descripción breve de las bases

mtcars

Contiene información de automóviles, como consumo de combustible, peso, potencia y número de cilindros.

cars

Contiene información sobre velocidad y distancia de frenado de automóviles.

iris

Contiene mediciones de flores de tres especies distintas.

airquality

Contiene mediciones de calidad del aire en Nueva York, incluyendo ozono, radiación solar, viento y temperatura.

3. Objetivo general

Diseñar un sistema de auditoría automática que reciba varias bases de datos conocidas y genere reportes resumen usando herramientas de iteración en R.

4. Objetivos específicos

1. Crear funciones que identifiquen columnas numéricas y no numéricas.
2. Calcular medidas resumen para todas las columnas numéricas de varias bases de datos.
3. Detectar columnas con valores faltantes.
4. Aplicar funciones de manera segura usando manejo de errores.
5. Comparar pares de variables numéricas usando `map2()`.
6. Combinar reportes parciales usando `reduce()`.
7. Construir una tabla final que resuma el estado de cada base de datos.

5. Paquetes permitidos

Se recomienda cargar solo estos paquetes:

```
library(purrr)
library(dplyr)
```

```
##
## Adjuntando el paquete: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union

library(tibble)
```

6. Estructura del proyecto

El proyecto se divide en seis partes.

Parte 1. Preparación de las bases de datos

Crea una lista llamada **bases** que contenga las 4 bases de datos.

Luego verifica:

1. Cuántas filas tiene cada base.
2. Cuántas columnas tiene cada base.
3. Qué clase tiene cada base.

```
bases <- list(mtcars,cars,iris,airquality)
names(bases) <- c("mtcars","cars","iris","airquality")

columnas_bases <- map_int(bases,ncol)
filas_bases <- map_int(bases,nrow)
clase_bases <- map_chr(bases,class)

tabla <- tibble(base=names(bases),
                columnas=columnas_bases,
                filas=filas_bases,
                clase=clase_bases)

tabla
```

```
## # A tibble: 4 x 4
##   base      columnas filas clase
##   <chr>      <int> <int> <chr>
## 1 mtcars      11     32 data.frame
## 2 cars         2     50 data.frame
## 3 iris         5    150 data.frame
## 4 airquality   6    153 data.frame
```

Parte 2. Auditoría de tipos de variables

Para cada base de datos, identifica:

1. Cuántas columnas son numéricas.
2. Cuántas columnas no son numéricas.
3. Si todas las columnas son numéricas.
4. Si existe al menos una columna no numérica.

```
numericas <- function(dato){
  a<- keep(map_lgl(dato,is.numeric),isTRUE)
  salida <- length(a)
  salida
}

no_numericas <- function(dato){
  a<- keep(map_lgl(dato,is.numeric),isFALSE)
  salida <- length(a)
  salida
}

todas <- function(dato){
  a<- map_lgl(dato,is.numeric)
  every(a,isTRUE)
}

alguna <- function(dato){
  a<- map_lgl(dato,is.numeric)
  some(a,isFALSE)
}

columnas_numericas <- map_int(bases,numericas)
columnas_no_numericas <- map_int(bases,no_numericas)
Todas_numericas <- map_lgl(bases,todas)
Almenos_una_no_numerica <- map_lgl(bases,alguna)

tabla2 <- tibble(base=names(bases),
                 numericas = columnas_numericas,
                 no_numericas=columnas_no_numericas,
                 Todas_numericas=Todas_numericas,
                 Almenos_una_no_numerica=Almenos_una_no_numerica)

tabla2
```

```
## # A tibble: 4 x 5
##   base      numericas no_numericas Todas_numericas Almenos_una_no_numerica
##   <chr>         <int>         <int> <lgl>              <lgl>
## 1 mtcars          11             0 TRUE              FALSE
## 2 cars            2             0 TRUE              FALSE
## 3 iris            4             1 FALSE             TRUE
## 4 airquality      6             0 TRUE              FALSE
```

Parte 3. Reporte automático de columnas numéricas

Para cada base de datos, conserva únicamente las columnas numéricas.

Después, para cada columna numérica calcula:

1. media;
2. mediana;
3. desviación estándar;
4. mínimo;
5. máximo;
6. rango.

Debes ignorar valores faltantes usando `na.rm = TRUE`.

```
numericas <- function(base){
  keep(base,is.numeric)
}
rango <- function(x,na.rm=TRUE){
  max(x) - min(x)
}
bases_numericas <- map(bases,numericas)

funciones <- list(media=mean,
                  mediana=median,
                  desviacion_estandar=sd,
                  minimo=min,
                  maximo=max,
                  rango=rango)

funcion1 <- function(base,funciones,na.rm = TRUE){
  resultado <- list()
  for (i in seq_along(funciones)){
    resultado[[i]] <- map(base,funciones[[i]],na.rm = TRUE)
  }
  names(resultado)<- names(funciones)
  resultado
}

resultado3 <- map(bases_numericas,~funcion1(.,funciones))

mtcars_tabla <- tibble(
  variable = names(bases_numericas$mtcars),
  media = unlist(resultado3$mtcars$media),
  mediana = unlist(resultado3$mtcars$mediana),
  desviacion_estandar = unlist(resultado3$mtcars$desviacion_estandar),
  minimo = unlist(resultado3$mtcars$minimo),
  maximo = unlist(resultado3$mtcars$maximo),
  rango = unlist(resultado3$mtcars$rango))

cars_tabla <- tibble(
  variable = names(bases_numericas$cars),
  media = unlist(resultado3$cars$media),
```

```

mediana = unlist(resultado3$cars$mediana),
desviacion_estandar = unlist(resultado3$cars$desviacion_estandar),
minimo = unlist(resultado3$cars$minimo),
maximo = unlist(resultado3$cars$maximo),
rango = unlist(resultado3$cars$rango))

iris_tabla <- tibble(
  variable = names(bases_numericas$iris),
  media = unlist(resultado3$iris$media),
  mediana = unlist(resultado3$iris$mediana),
  desviacion_estandar = unlist(resultado3$iris$desviacion_estandar),
  minimo = unlist(resultado3$iris$minimo),
  maximo = unlist(resultado3$iris$maximo),
  rango = unlist(resultado3$iris$rango))

airquality_tabla <- tibble(
  variable = names(bases_numericas$airquality),
  media = unlist(resultado3$airquality$media),
  mediana = unlist(resultado3$airquality$mediana),
  desviacion_estandar = unlist(resultado3$airquality$desviacion_estandar),
  minimo = unlist(resultado3$airquality$minimo),
  maximo = unlist(resultado3$airquality$maximo),
  rango = unlist(resultado3$airquality$rango))

mtcars_tabla

```

```

## # A tibble: 11 x 7
##   variable  media mediana desviacion_estandar minimo maximo  rango
##   <chr>    <dbl>   <dbl>           <dbl>   <dbl> <dbl> <dbl>
## 1 mpg      20.1    19.2             6.03    10.4  33.9  23.5
## 2 cyl       6.19     6             1.79     4     8     4
## 3 disp     231.    196.           124.    71.1  472   401.
## 4 hp       147.    123            68.6    52    335   283
## 5 drat      3.60    3.70            0.535    2.76  4.93  2.17
## 6 wt        3.22    3.32            0.978    1.51  5.42  3.91
## 7 qsec     17.8    17.7            1.79    14.5  22.9  8.4
## 8 vs        0.438    0             0.504    0     1     1
## 9 am        0.406    0             0.499    0     1     1
## 10 gear     3.69     4             0.738    3     5     2
## 11 carb     2.81     2             1.62    1     8     7

```

```
cars_tabla
```

```

## # A tibble: 2 x 7
##   variable media mediana desviacion_estandar minimo maximo rango
##   <chr>    <dbl>   <dbl>           <dbl>   <dbl> <dbl> <dbl>
## 1 speed    15.4    15             5.29     4    25    21
## 2 dist     43.0    36            25.8     2   120   118

```

```
iris_tabla
```

```
## # A tibble: 4 x 7
##   variable      media mediana desviacion_estandar minimo maximo rango
##   <chr>         <dbl>   <dbl>          <dbl>   <dbl>   <dbl> <dbl>
## 1 Sepal.Length  5.84    5.8            0.828    4.3     7.9   3.6
## 2 Sepal.Width   3.06    3              0.436    2       4.4   2.4
## 3 Petal.Length  3.76    4.35           1.77     1       6.9   5.9
## 4 Petal.Width   1.20    1.3            0.762    0.1     2.5   2.4
```

```
airquality_tabla
```

```
## # A tibble: 6 x 7
##   variable      media mediana desviacion_estandar minimo maximo rango
##   <chr>         <dbl>   <dbl>          <dbl>   <dbl>   <dbl> <dbl>
## 1 Ozone        42.1    31.5           33.0     1     168    NA
## 2 Solar.R      186.    205           90.1     7     334    NA
## 3 Wind          9.96    9.7            3.52    1.7    20.7   19
## 4 Temp         77.9    79             9.47    56     97    41
## 5 Month         6.99    7              1.42    5       9     4
## 6 Day          15.8    16             8.86    1      31    30
```

Parte 4. Manejo de errores en auditoría

En la industria, las bases de datos no siempre vienen limpias. Algunas columnas pueden tener texto, valores faltantes o estructuras inesperadas.

Crea una función llamada `media_segura()` que intente calcular la media de una columna. Si no se puede calcular, debe devolver NA.

Luego aplícala a todas las columnas de cada base de datos, no solo a las numéricas.

Ejemplo conceptual

Si se aplica a `iris`, las columnas numéricas deben devolver una media, pero la columna `Species` debe devolver NA.

```
media_segura <- safely(mean)
funcion2<- function(base){
  map(base,media_segura,na.rm = TRUE)
}
resultado4 <-map(bases,funcion2)
```

```
## Warning in mean.default(...): argument is not numeric or logical: returning NA
```

```
mtcars_seguro <- tibble(
  variable = names(mtcars),
  media = unlist(resultado4$mtcars))

cars_seguro <- tibble(
  variable = names(cars),
  media = unlist(resultado4$cars))

iris_seguro <- tibble(
  variable = names(iris),
  media = unlist(resultado4$iris))

airquality_seguro <- tibble(
  variable = names(airquality),
  media = unlist(resultado4$airquality))
```

```
mtcars_seguro
```

```
## # A tibble: 11 x 2
##   variable  media
##   <chr>    <dbl>
## 1 mpg      20.1
## 2 cyl       6.19
## 3 disp     231.
## 4 hp      147.
## 5 drat      3.60
## 6 wt        3.22
## 7 qsec     17.8
## 8 vs        0.438
## 9 am        0.406
## 10 gear     3.69
## 11 carb     2.81
```

```
cars_seguro
```

```
## # A tibble: 2 x 2
##   variable media
##   <chr>    <dbl>
## 1 speed    15.4
## 2 dist     43.0
```

```
iris_seguro
```

```
## # A tibble: 5 x 2
##   variable      media
##   <chr>        <dbl>
## 1 Sepal.Length  5.84
## 2 Sepal.Width   3.06
## 3 Petal.Length  3.76
## 4 Petal.Width   1.20
## 5 Species       NA
```



```
airquality_seguro
```

```
## # A tibble: 6 x 2
##   variable  media
##   <chr>    <dbl>
## 1 Ozone    42.1
## 2 Solar.R 186.
## 3 Wind     9.96
## 4 Temp    77.9
## 5 Month    6.99
## 6 Day     15.8
```

Parte 5. Comparación entre pares de variables

En algunos reportes empresariales se necesita comparar pares de variables.

Por ejemplo:

- en `cars`, comparar `dist` contra `speed`;
- en `iris`, comparar `Petal.Length` contra `Sepal.Length`;
- en `iris`, comparar `Petal.Width` contra `Sepal.Width`;
- en `mtcars`, comparar `hp` contra `mpg`;
- en `airquality`, comparar `Temp` contra `Wind`.

Crea una tabla llamada `comparaciones` con esta estructura:

```
comparaciones <- tibble(
  base = c("cars", "iris", "iris", "mtcars", "airquality"),
  variable_1 = c("dist", "Petal.Length", "Petal.Width", "hp", "Temp"),
  variable_2 = c("speed", "Sepal.Length", "Sepal.Width", "mpg", "Wind"))
comparaciones
```

```
## # A tibble: 5 x 3
##   base      variable_1 variable_2
##   <chr>    <chr>      <chr>
## 1 cars     dist       speed
## 2 iris     Petal.Length Sepal.Length
## 3 iris     Petal.Width  Sepal.Width
## 4 mtcars   hp         mpg
## 5 airquality Temp      Wind
```

Para cada fila, calcula:

$$\text{diferencia de medias} = \bar{x}_1 - \bar{x}_2.$$

También calcula:

$$\text{razón de rangos} = \frac{\text{rango}(x_1)}{\text{rango}(x_2)}.$$

```
comparaciones <- tibble(
  base = c("cars", "iris", "iris", "mtcars", "airquality"),
  variable_1 = c("dist", "Petal.Length", "Petal.Width", "hp", "Temp"),
  variable_2 = c("speed", "Sepal.Length", "Sepal.Width", "mpg", "Wind"),

  diferencia_medias = c(mean(cars$dist) - mean(cars$speed),
                        mean(iris$Petal.Length) - mean(iris$Sepal.Length),
                        mean(iris$Petal.Width) - mean(iris$Sepal.Width),
                        mean(mtcars$hp) - mean(mtcars$mpg),
                        mean(airquality$Temp) - mean(airquality$Wind)),

  razon_de_rangos= c(rango(cars$dist) / rango(cars$speed),
                    rango(iris$Petal.Length) / rango(iris$Sepal.Length),
                    rango(iris$Petal.Width) / rango(iris$Sepal.Width),
                    rango(mtcars$hp) / rango(mtcars$mpg),
                    rango(airquality$Temp) / rango(airquality$Wind)
  ))

comparaciones
```

```
## # A tibble: 5 x 5
##   base      variable_1 variable_2  diferencia_medias razon_de_rangos
##   <chr>      <chr>      <chr>            <dbl>          <dbl>
## 1 cars      dist        speed             27.6           5.62
## 2 iris      Petal.Length Sepal.Length      -2.09           1.64
## 3 iris      Petal.Width  Sepal.Width       -1.86           1
## 4 mtcars    hp          mpg              127.           12.0
## 5 airquality Temp        Wind              67.9           2.16
```

Parte 6. Reporte final usando reduce()

Al final debes construir un reporte general que combine varias tablas:

1. tabla de dimensiones de cada base;
2. tabla de tipos de variables;
3. tabla de cantidad de valores faltantes;
4. tabla resumen de columnas numéricas.

El objetivo es obtener una tabla final donde cada base tenga información general sobre su calidad.

```
#1 tabla de dimensiones de cada base
resultado7<- tibble(base=names(bases),
                    columnas = map_int(bases, ncol),
                    filas= map_int(bases,nrow))

resultado7
```

```
## # A tibble: 4 x 3
##   base      columnas filas
```

```
##   <chr>          <int> <int>
## 1 mtcars         11    32
## 2 cars           2     50
## 3 iris           5    150
## 4 airquality     6    153
```

```
resultado8 <- select(tabla,-clase)
resultado8
```

```
## # A tibble: 4 x 3
##   base      columnas filas
##   <chr>      <int> <int>
## 1 mtcars         11    32
## 2 cars           2     50
## 3 iris           5    150
## 4 airquality     6    153
```

#2 tabla de tipos de variables

```
funcion3<- function(dato,nombre){
  vector <- c()
  for (i in seq_along(dato)){
    vector[i] <- nombre
  }
  vector
}
tabla9 <- tibble(variable = unlist(map(bases,names)),
                  base = unlist(map2(bases,names(bases),funcion3)),
                  Tipos_de_variable = unlist(map(bases,~map_chr(.x,class))))
tabla9
```

```
## # A tibble: 24 x 3
##   variable base  Tipos_de_variable
##   <chr>    <chr> <chr>
## 1 mpg      mtcars numeric
## 2 cyl      mtcars numeric
## 3 disp     mtcars numeric
## 4 hp       mtcars numeric
## 5 drat     mtcars numeric
## 6 wt       mtcars numeric
## 7 qsec     mtcars numeric
## 8 vs       mtcars numeric
## 9 am       mtcars numeric
## 10 gear    mtcars numeric
## # i 14 more rows
```

#3 tabla de cantidad de valores faltantes

```
funcion4 <- function(base,variable){
  base <- get(base)

  sum(is.na(base[[variable]]))
}
```

```
tabla10 <- tabla9 %>%
  mutate(faltantes = unlist(map2(base,variable,funcion4)))
```

```
tabla10
```

```
## # A tibble: 24 x 4
##   variable base   Tipos_de_variable faltantes
##   <chr>    <chr> <chr>                <int>
## 1 mpg      mtcars numeric              0
## 2 cyl      mtcars numeric              0
## 3 disp     mtcars numeric              0
## 4 hp       mtcars numeric              0
## 5 drat     mtcars numeric              0
## 6 wt       mtcars numeric              0
## 7 qsec     mtcars numeric              0
## 8 vs       mtcars numeric              0
## 9 am       mtcars numeric              0
## 10 gear    mtcars numeric              0
## # i 14 more rows
```

```
#4 tabla resumen de columnas numéricas
```

```
tabla11 <- filter(tabla10,Tipos_de_variable %in% c("numeric","integer" ))
tabla11
```

```
## # A tibble: 23 x 4
##   variable base   Tipos_de_variable faltantes
##   <chr>    <chr> <chr>                <int>
## 1 mpg      mtcars numeric              0
## 2 cyl      mtcars numeric              0
## 3 disp     mtcars numeric              0
## 4 hp       mtcars numeric              0
## 5 drat     mtcars numeric              0
## 6 wt       mtcars numeric              0
## 7 qsec     mtcars numeric              0
## 8 vs       mtcars numeric              0
## 9 am       mtcars numeric              0
## 10 gear    mtcars numeric              0
## # i 13 more rows
```

```
#funciones seguras
```

```
media_segura <- safely(mean, otherwise = NA_real_)
mediana_segura <- safely(median, otherwise = NA_real_)
desviacion_estandar_segura <- safely(sd, otherwise = NA_real_)
minimo_seguro <- safely(min, otherwise = NA_real_)
maximo_seguro <- safely(max, otherwise = NA_real_)
rango_seguro <- safely(rango, otherwise = NA_real_)
```

```
tabla12 <- tabla11 %>%
  mutate(media=map2_dbl(base,variable, ~ unlist(media_segura(get(.x)[[.y]]))),
         mediana=map2_dbl(base,variable, ~ unlist(mediana_segura(get(.x)[[.y]]))),
         desviacion_estandar=map2_dbl(base,variable, ~ unlist(desviacion_estandar_segura(get(.x)[[.y]]))))
```

```

minimo=map2_dbl(base,variable, ~ unlist(minimo_seguro(get(.x)[[.y]]))),
maximo=map2_dbl(base,variable, ~ unlist(maximo_seguro(get(.x)[[.y]]))),
rango=map2_dbl(base,variable, ~ unlist(rango_seguro(get(.x)[[.y]])))
tabla12

```

```

## # A tibble: 23 x 10
##   variable base   Tipos_de_variable faltantes   media mediana
##   <chr>   <chr>   <chr>             <int>   <dbl>   <dbl>
## 1 mpg     mtcars numeric             0    20.1    19.2
## 2 cyl     mtcars numeric             0     6.19     6
## 3 disp    mtcars numeric             0   231.    196.
## 4 hp      mtcars numeric             0   147.    123
## 5 drat    mtcars numeric             0    3.60    3.70
## 6 wt      mtcars numeric             0    3.22    3.32
## 7 qsec    mtcars numeric             0   17.8    17.7
## 8 vs      mtcars numeric             0    0.438     0
## 9 am      mtcars numeric             0    0.406     0
## 10 gear   mtcars numeric             0    3.69     4
## # i 13 more rows
## # i 4 more variables: desviacion_estandar <dbl>, minimo <dbl>, maximo <dbl>,
## #   rango <dbl>

```

7. Preguntas de interpretación

Responde brevemente:

1. ¿Qué base tiene más valores faltantes?

R// **airquality** con 44 valores faltantes, 37 en la variable **Ozone** y 7 en la variable **Solar.R**

2. ¿Qué base tiene más columnas numéricas?

```

tabla13 <- tabla12 %>%
  group_by(base) %>%
  summarise(variables_numericas = sum(Tipos_de_variable %in% c("numeric","integer")))
tabla13

```

```

## # A tibble: 4 x 2
##   base       variables_numericas
##   <chr>             <int>
## 1 airquality         6
## 2 cars                2
## 3 iris               4
## 4 mtcars            11

```

R// Como podemos ver en la tabla anterior **mtcars** tiene mas variables numericas con 11

3. ¿Qué variable tiene el mayor rango en todo el proyecto?

```
tabla14 <- tabla12 %>%
  filter(rango == max(rango, na.rm = TRUE))
tabla14
```

```
## # A tibble: 1 x 10
##   variable base   Tipos_de_variable faltantes media mediana desviacion_estandar
##   <chr>      <chr> <chr>                <int> <dbl>   <dbl>             <dbl>
## 1 disp      mtcars numeric                0  231.   196.             124.
## # i 3 more variables: minimo <dbl>, maximo <dbl>, rango <dbl>
```

R// A base de la tabla anterior la variable con mayor rango es disp de la base mtcars con un rango de 400.9

4. ¿Hay alguna base cuyas columnas sean todas numéricas?

R// las siguientes tablas son todas numericas: mtcars,cars, airquality. iris es la unica que no es toda numerica ya que tiene la variable species

5. ¿Qué comparación tiene la mayor diferencia de medias?

R// la comparacion entre hp y mpg de la base de datos mtcars con la diferencia de 126.596875

8. Comentario final

Este proyecto simula una tarea real de la industria: construir un sistema básico de auditoría de datos.

Antes de aplicar modelos avanzados, las empresas necesitan saber:

- qué tipo de variables tienen;
- si existen valores faltantes;
- qué variables son numéricas;
- cuáles tienen rangos grandes;
- qué bases están listas para análisis;
- dónde hay posibles problemas de calidad.